Code Review

- 프로젝트 뼈대 설계
- DB설계 & 전체 테이블 생성
- 메인 페이지 구성
- 관리자 대시보드 구현
- AI 비교분석 기능 구현

발표자 : 김도하

프로젝트 뼈대 설계

폴더 구조

```
AI-accident-detection C:₩Users₩jihun2240
  .venv
  ai
  app
    api
    common
    models
    repositories
    schemas
    services
    socket_events
    static
    templates
    __init__.py
    config.py
    extensions.py
  migrations
  .env
  .gitignore
  README.md
  requirements.txt
  run.py
```

ai/

AI 모델, 추론, 분석 관련 코드 모음

app/

Flask 웹 애플리케이션 핵심 영역

migrations/

Flask-Migrate/Alembic 기반 DB 마이그레이션 관리

run.py

프로젝트 실행 진입점

requirements.txt

패키지 의존성 관리

.env

환경변수 관리

app/

- **api/** : 요청 처리 및 라우팅
- **services/** : 핵심 비즈니스 로직
- **repositories/** : DB 접근 로직 분리
- **models/** : DB 테이블 구조 정의
- **schemas/** : 요청/응답 데이터 구조
- **templates / static/** : 화면 및 정적 리소스
- **socket_events/** : 실시간 알림 이벤트 처리
- **__init__.py / config.py / extensions.py** : 앱 초기화 및 환경 설정

블루프린트 분리

__init__.py

```python
# ----------------------------
# 블루프린트 등록
# ----------------------------
from app.api.auth_routes import auth_bp
from app.api.main_routes import main_bp
from app.api.report_route import report_bp
from app.api.admin_routes import admin_bp
from app.api.admin_report_routes import admin_report_bp
from app.api.admin_user_routes import admin_user_bp
from app.api.admin_ai_routes import admin_ai_bp
from app.api.report_list_routes import report_list_bp
from app.api.admin_realtime_alert_routes import admin_realtime_alert_bp
from app.api.realtime_monitor_routes import realtime_monitor_bp
from app.api.kakao_navigation_routes import kakao_navigation_bp
from app.api.admin_region_analysis_routes import admin_region_analysis_bp
```

```python
app.register_blueprint(auth_bp)
app.register_blueprint(main_bp)
app.register_blueprint(report_bp)
app.register_blueprint(admin_bp)
app.register_blueprint(admin_report_bp)
app.register_blueprint(admin_user_bp)
app.register_blueprint(admin_ai_bp)
app.register_blueprint(report_list_bp)
app.register_blueprint(admin_realtime_alert_bp)
app.register_blueprint(realtime_monitor_bp)
app.register_blueprint(kakao_navigation_bp)
app.register_blueprint(admin_region_analysis_bp)
```

기능별 블루프린트를 분리하고 앱 초기화 단계에서 일괄 등록하여, 확장성과 유지보수성을 높인 구조입니다.

서비스 / 리포지토리 / 모델 구조

Service/
실제 기능 처리와 비즈니스 로직 담당

services
- __init__.py
- admin_ai_compare_service.py
- admin_ai_service.py
- admin_region_analysis_service.py
- admin_report_service.py
- admin_role_request_service.py
- admin_service.py
- admin_user_service.py
- auth_service.py
- dashboard_service.py
- llm_service.py
- realtime_alert_service.py
- realtime_monitor_service.py
- report_list_service.py
- report_service.py
- yolo_service.py

Repository/
DB 조회, 저장, 수정 등 데이터 접근 담당

repositories
- __init__.py
- admin_region_analysis_repository.py
- ai_compare_repository.py
- detection_repository.py
- realtime_alert_repository.py
- realtime_monitor_repository.py
- report_repository.py
- role_request_repository.py
- user_repository.py

Model /
DB 테이블 구조와 객체 관계 정의

models
- __init__.py
- admin_log_model.py
- ai_compare_result.py
- ai_compare_run.py
- alert_model.py
- detection_model.py
- report_file_model.py
- report_model.py
- report_status_log_model.py
- role_request_model.py
- user_model.py

비즈니스 로직, DB 접근, 데이터 구조를 계층별로 분리해 책임을 명확히 한 구조

공통 레이아웃 구조

templates/layout
공통 화면 요소를 템플릿 단위로 분리한 구조

- templates
  - layout
    - _team_sidebar.html
    - footer.html
    - header.html

static/css
공통 스타일 + 화면별 스타일 분리 구조.

- css
  - account_manage.css
  - admin_ai.css
  - admin_common.css
  - admin_dashboard.css
  - admin_region_analysis.css
  - admin_reports.css
  - admin_users.css
  - auth.css
  - common.css
  - main.css
  - my_reports.css
  - realtime_alert.css
  - realtime_monitor.css
  - report.css
  - report_detail.css
  - report_edit.css

static/js
공통 동작 + 화면별 동작 분리 구조.

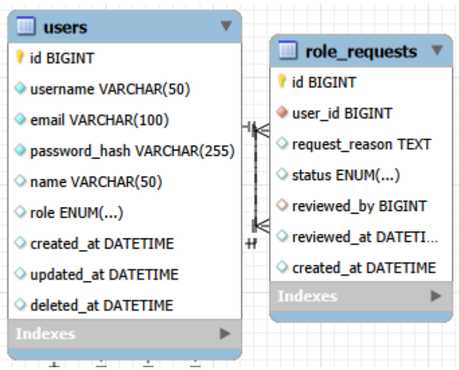
- js
  - admin_dashboard.js
  - admin_region_analysis.js
  - admin_report_detail.js
  - auth.js
  - common.js
  - my_report_detail.js
  - my_report_edit.js
  - my_reports.js
  - realtime_alert.js
  - realtime_monitor.js
  - report.js
  - role_request.js

공통 레이아웃은 템플릿으로 재사용하고, 페이지별 CSS·JS를 분리해 일관성과 유지보수성을 함께 확보한 구조

DB 설계 및 기능 연결 구조

회원/권한 관리부터 신고, 파일, 탐지 결과 저장까지의 DB 흐름

사용자 & 권한 흐름



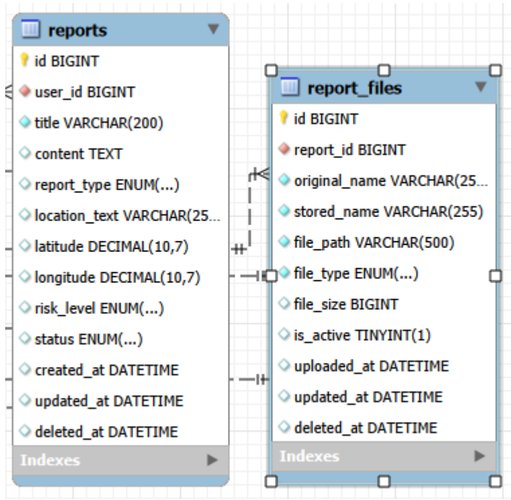
핵심 테이블

- Users
- role_requests

흐름

1. 사용자가 회원가입 → users 저장
2. 관리자가 되려면 → role_requests 생성
3. 관리자가 승인하면 → users.role = admin 변경

신고 기능



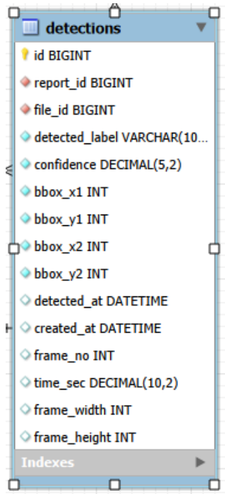
핵심 테이블

- Reports
- report_files

흐름

1. 사용자가 신고 작성 → reports 저장
2. 이미지/영상 업로드 → report_files 저장
3. 하나의 신고에 여러 파일 가능

AI 탐지 흐름



핵심 테이블

- detections

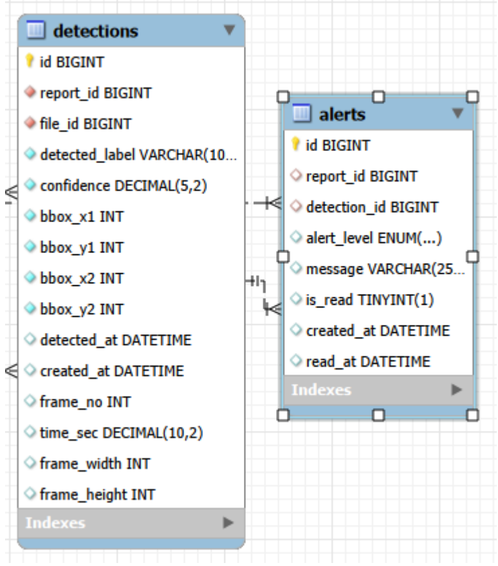
흐름

1. 업로드된 파일 기준으로 AI 분석 실행
2. 탐지 결과를 detections 테이블에 저장
3. 저장 정보: bbox 좌표, confidence, 프레임 정보

DB 설계 및 기능 연결 구조

탐지 결과를 운영 기능과 비교분석 기능으로 확장한 DB 구조

실시간 알림 기능



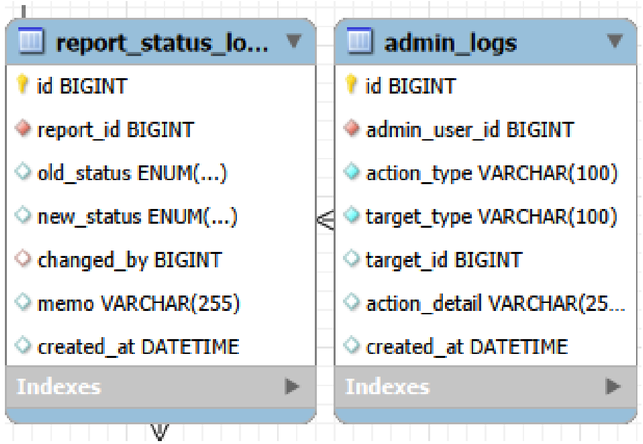
핵심 테이블

- alerts

흐름

1. 위험 탐지 발생
2. detections 기반으로 알림 생성
3. 관리자에게 전달

관리자 처리 흐름



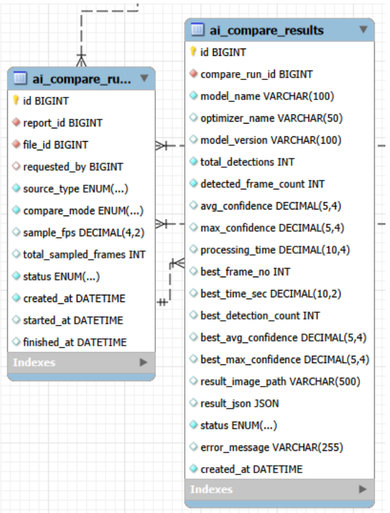
핵심 테이블

- report_status_logs
- admin_logs

흐름

1. 관리자가 신고 상태 변경
2. 변경 이력 저장 → report_status_logs
3. 관리자 행동 기록 → admin_logs

비교분석



핵심 테이블

- ai_compare_runs
- ai_compare_results

흐름

1. 관리자 → 비교분석 실행
2. 실행 단위 저장 → ai_compare_runs
3. 모델별 결과 저장 → ai_compare_results

관리자 대시보드 서비스 로직

관리자 대시보드에 필요한 통계와 목록 데이터를 개별 메서드로 조회한 뒤, 최종적으로 하나의 응답 구조로 묶어 반환하도록 설계했습니다.

코드리뷰

```python
def get_dashboard_data(self) -> dict:
    """
    대시보드 전체 데이터를 조회하여 딕셔너리 형태로 반환한다.
    """
    total_reports = self.get_total_reports_count()
    completed_reports = self.get_completed_reports_count()
    pending_reports = self.get_pending_reports_count()
    false_reports = self.get_false_reports_count()
    recent_reports = self.get_recent_reports(limit=5)
    pending_role_requests = self.get_pending_role_requests(limit=5)

    return {
        "total_reports": total_reports,
        "completed_reports": completed_reports,
        "pending_reports": pending_reports,
        "false_reports": false_reports,
        "recent_reports": recent_reports,
        "pending_role_requests": pending_role_requests,
    }
```

이 메서드는 관리자 대시보드에 필요한 통계와
목록 데이터를 각각 조회한 뒤,
하나의 딕셔너리로 묶어서 반환하는 역할을 합니다.

```python
def get_pending_reports_count(self) -> int:  1 usage
    """
    미처리 신고 수 조회
    기준:
    - 접수
    - 확인중
    """
    return Report.query.filter(
        Report.status.in_(["접수", "확인중"]),
        Report.deleted_at.is_(None)
    ).count()
```

미처리 신고는 접수와 확인중 상태를 포함하도록 정의했고,
삭제된 데이터는 제외해서 실제 운영 데이터만 집계했습니다.

```python
def get_pending_role_requests(self, limit: int = 5) -> list[dict]:  1 usage
    """
    대기 중인 관리자 권한 신청 목록 조회

    User 테이블과 조인하여 신청자 이름도 함께 가져온다.
    """
    rows = (
        db.session.query(RoleRequest, User)
        .join(User, RoleRequest.user_id == User.id)
        .filter(RoleRequest.status == "대기")
        .order_by(RoleRequest.created_at.desc())
        .limit(limit)
        .all()
    )
```

권한 신청 목록은 RoleRequest와 User를 조인해서 신청자 이름까지
함께 조회하도록 구현했습니다.

관리자 AI 비교분석 기능 구현

동일한 신고 데이터에 대해 3개의 AI 모델을 순차 실행하고,
결과를 가공·저장하여 관리자 화면에서 비교할 수 있도록 구현했습니다.

아키텍처 흐름

사용자 → Flask Controller → Service Layer → SocketIO 비동기 실행 → AI 모델 3개 실행 → 결과 가공 (confidence / class count) → DB 저장/프론트

핵심 기능

비동기 처리 | 모델 비교 | 결과 시각화

코드리뷰

AI 모델 결과를 비교 가능한 데이터로 만드는 핵심 코드

이 코드는 비교분석의 핵심 로직으로, 3개의 AI 모델을 동일한 데이터에 대해 순차적으로 실행합니다.

각 모델의 탐지 결과를 total detections, confidence, 클래스별 탐지 수 형태로 가공한 뒤, 이를 DB에 저장하여 관리자 화면에서 모델 간 성능을 비교할 수 있도록 구현했습니다.”

모델 반복 실행 (for문)

모델 분석 실행

```
for model_config in cls.MODEL_CONFIGS:
    try:
        print("=" * 60)
        print("MODEL_NAME:", model_config["model_name"])
        print("MODEL_VERSION:", model_config["model_version"])

        result_data = cls.execute_model_analysis(
            report=report,
            report_file=report_file,
            model_config=model_config
        )

        if (
            result_data.get("sampled_frames") is not None
            and sampled_frames_for_run is None
        ):
            sampled_frames_for_run = result_data.get("sampled_frames")

        print(f"🔥 저장 시도: {model_config['model_name']}")

        safe_result_json = result_data.get("result_json")
        if safe_result_json is not None:
            safe_result_json = json.loads(json.dumps(safe_result_json))
```

DB 저장

```
AiCompareRepository.create_result(
    compare_run_id=compare_run_id,
    model_name=result_data["model_name"],
    optimizer_name=result_data.get("optimizer_name"),
    model_version=result_data.get("model_version"),
    total_detections=result_data.get("total_detections", 0),
    detected_frame_count=result_data.get("detected_frame_count", 0),
    avg_confidence=result_data.get("avg_confidence"),
    max_confidence=result_data.get("max_confidence"),
    processing_time=result_data.get("processing_time"),
    best_frame_no=result_data.get("best_frame_no"),
    best_time_sec=result_data.get("best_time_sec"),
    best_detection_count=result_data.get("best_detection_count"),
    best_avg_confidence=result_data.get("best_avg_confidence"),
    best_max_confidence=result_data.get("best_max_confidence"),
    result_image_path=result_data.get("result_image_path"),
    result_json=safe_result_json,
    status=result_data.get("status", "완료"),
    error_message=result_data.get("error_message")
)
```

Code Review

- AI 분석 및 신고 텍스트 자동화
- 위치 기반 데이터 전처리
- 시스템 연동 및 데이터 정합성 설계

발표자 : 송명근

AI 분석 및 신고 텍스트 자동화

- **자동화**: 탐지된 객체(낙하물, 사고 등) 정보를 기반으로 신고 제목과 상세 내용을 AI가 자동 생성하여 관제 효율성 극대화.
- **장애 내성 설계**: API 할당량 초과나 네트워크 오류 발생 시에도 시스템이 중단되지 않도록 **Fallback 로직**을 구현하여 서비스 안정성 확보.
- **데이터 매핑**: AI의 영문 라벨을 한국어 표준 데이터로 변환하여 사용자에게 직관적인 정보 제공.

```
# llm_service.py: 장애 내성을 위한 Fallback 로직
try:
    response = client.models.generate_content(
        model="gemini-1.5-flash",
        contents=prompt,
        config=types.GenerateContentConfig(response_mime_type="application/json")
    ) # Gemini API 호출
    return json.loads(response.text)


# API 타임아웃 또는 할당량 초과 시 시스템 중단을 막는 Fallback 처리
except google_exceptions.DeadlineExceeded:
    logger.error("🎉 [LLM 타임아웃] API 응답 시간 초과")
    return fallback_result
except google_exceptions.ResourceExhausted:
    logger.error("🎉 [LLM 할당량 초과] API 사용량 한도 도달")
    return fallback_result

# API 타임아웃 또는 할당량 초과 시 미리 정의된 기본 문구(Fallback) 반환
except google_exceptions.DeadlineExceeded:
    logger.error("🎉 [LLM 타임아웃] API 응답 시간 초과")
    return fallback_result
except google_exceptions.ResourceExhausted:
    logger.error("🎉 [LLM 할당량 초과] API 사용량 한도 도달")
    return fallback_result
```

핵심 키워드

- 신고문 자동 생성
- 외부 API 예외 처리
- fallback 설계
- 서비스 연속성

위치 기반 데이터 전처리

- **Exif 분석**: 업로드된 이미지/영상에서 위경도 좌표를 추출하여 지도상에 사고 발생 위치를 자동으로 매핑.
- **파일 검증**: 대용량 미디어 파일(50MB 제한) 필터링 및 확장자 체크를 통해 서버 자원 보호 및 보안 강화.

```
report_service.py: 서버 리소스 보호를 위한 파일 검증
    class ReportService:
        MAX_FILE_SIZE = 50 * 1024 * 1024 # 50MB 제한
  @staticmethod
    def process_report_submission(user_id, form_data, upload_file):
      # 파일 크기 및 확장자 유효성 검사
      if file_size > ReportService.MAX_FILE_SIZE:
         raise ValueError(f"파일 용량은 50MB를 넘을 수 없습니다.")

      if ext not in ReportService.ALLOWED_EXTENSIONS:
        raise ValueError(f"허용되지 않는 파일 형식입니다: {ext}")
```

핵심 키워드

- **메타데이터 추출**
- **위치 정보 자동 매핑**
- **파일 크기 제한**
- **서버 자원 보호**

시스템 연동 및 데이터 정합성 설계

- **Atomic Operation 지향**: 데이터베이스 저장 과정에서 예외 발생 시, `db.session.rollback()` 을 통해 데이터 일관성을 유지.
- **물리 파일 자동 삭제**: DB 저장이 실패할 경우, 서버에 이미 업로드된 이미지/영상 파일을 `os.remove()` 로 즉시 삭제하여 서버 저장 공간 낭비(Orphan Files)를 원천 차단.
- **보안 및 안정성**: 트라이-익셉트(try-except) 구조를 통해 예상치 못한 런타임 에러에도 서버가 다운되지 않고 적절한 에러 메시지를 반환하도록 설계.

```
# report_service.py: 예외 발생 시 데이터 및 리소스 복구 로직
try:
    db.session.add(new_report)
    db.session.commit() # 최종 확정

except Exception as e:
    db.session.rollback() # DB 상태를 이전으로 되돌림

    # DB 저장 실패 시 서버에 남은 쓸모없는 물리 파일까지 삭제하여 리소스 보호
    if saved_file_path and os.path.exists(saved_file_path):
        os.remove(saved_file_path)

    logger.error(f"🚨 [신고 접수 실패] 리소스 복구 완료: {e}")
    raise e
```

핵심 키워드

- commit 이후 후속 처리
- Rollback
- 물리 파일 정리
- 데이터 정합성
- 리소스 보호